

HRMS — Code + Redis Behaviour (Feature-by-Feature Flow)

This document does **not** explain what Redis or BullMQ are. It traces, for **every HRMS feature that touches Redis**, exactly what **our code** does step-by-step: what we write to Postgres, what we put in Redis, what the screen shows, what happens when the Redis entry's **TTL expires / the job is removed**, and whether the code then **falls back to a database call**. Source-verified against the actual code (file paths/lines cited in each section).

1. How to read this — Redis plays exactly 3 roles in our code

Every Redis touch in HRMS is one of these three. Knowing which role a feature uses tells you instantly what "TTL expiry" means for it.

Role	What our code stores in Redis	Is it the source of truth?	What happens at TTL / removal
A. Job queue (BullMQ)	A transient <i>job</i> describing work to do (e.g. "import these rows", "compute this payroll run", "send this email").	No. The real record is a row in Postgres (<code>DataImport</code> , <code>PayRun</code> , <code>Invitation</code> , ...).	Job is auto-deleted ~1h after success / 2d after failure. The Postgres row remains; the screen reads Postgres.
B. Read-through cache (TTL)	A <i>copy</i> of a Postgres read (permissions, profile, unread count, leave policy) or a pre-computed snapshot (org-chart, compensation analytics, birthdays/anniversaries).	No. Always a copy of Postgres.	On expiry the next request misses Redis and re-runs the Postgres query (the loader), then re-caches. This is the DB fallback.
C. Pub/sub message bus	Nothing is stored. A one-shot message is published on a channel and forwarded to connected browsers (SSE).	n/a	n/a — it's a signal, not storage. The browser reacts by re-fetching from Postgres .

The single most important sentence for the reviewer: *No business data lives permanently in Redis.* Postgres is always the source of truth. Redis is either (A) a to-do list for the worker, (B) a short-lived cached copy that rebuilds from Postgres when it expires, or (C) a doorbell that tells the screen "go re-fetch from Postgres."

2. Direct answers to the two examples raised on the call

a) "The email trigger button — what does the code + Redis do?"

There is **no standalone 'send email' button**. Email is always a **side-effect of a domain action**. The button being referred to is most likely **Invite** (Settings → Users). Flow (Role A — job queue):

- Click **Invite** → `POST /api/v1/hrms/invitations`.
- Code **writes the `Invitation` row to Postgres first** (this is the real record).
- Code then **enqueues an `email job` to Redis** (BullMQ) — the job carries the to-address, subject, HTML.
- The **worker** picks the job up and sends it via SMTP (nodemailer).
- On success the job is **removed from Redis ~1h later** (`removeOnComplete`). The invitation/notification record stays in Postgres.
- Screen:** shows a success toast immediately; the Users list reflects the Postgres `Invitation` row. The email send itself is fire-and-forget.
- If Redis is down:** the single-invite path **falls back to sending inline** (so one invite still works); bulk/notification emails are skipped but the in-app notification row is still written. Full trace: **Flow B → Feature 1**.

b) "The anniversary item — it sits with a TTL, then is removed; does the code re-call the DB?"

Yes, this one is genuinely Redis-cached (Role B), and your observation is correct. Birthdays/anniversaries/celebrations are cached with a **300-second TTL** (`celebrations:{orgId}` , `birthdays:{orgId}` , `anniversaries:{orgId}`).

- The data is **computed from Postgres**, cached in Redis for 5 minutes, and served from cache on repeat loads.
- When the 300s TTL expires, the key disappears from Redis (what you saw).** The screen does **not** instantly change — TTL expiry does not push anything to the browser. Instead, **the next time the widget loads (page reload / tab refocus / React Query staleness), the code misses Redis and re-runs the Postgres query**, then re-caches for another 300s.
- So: **expiry ≠ screen update**. The DB re-call happens on the *next request*, not at the instant the key vanishes. Full trace: **Flow D → Part 3**.

3. Master map — every Redis-backed feature at a glance

Feature	Role	Redis key / queue	TTL / lifetime	In Redis	In Postgres (source of truth)	Screen update	DB fallback on expiry/miss
Bulk employee import	A	bull:bulk-employee-import:*	1h after done	the import job + rows	DataImport + created Employee rows	2s polling of DataImport	reads DataImport /employees from PG
Bulk bank import	—	(none)	—	—	inline Employee.update (synchronous)	synchronous response	n/a (never uses Redis)
Payroll run (compute)	A	bull:payroll-run:*	1h after done	job = {runId} only	PayRun.compute* cols + Payslip rows	React Query refetchInterval + SSE	reads PayRun / Payslip from PG
PDF (payslip release)	A	bull:pdf-generation:*	1h after done	the job; PDF as email attachment	Payslip row (PDF not stored)	toast	re-build from PG via resend
Email (invite/notify)	A	bull:email:*	1h after done	the email job	Invitation / Notification row	toast / list	n/a (email is transient)
Cron — ticket auto-close	A	scheduler in cron-tickets	repeatable	schedule only	Ticket rows	next load / SSE	all data in PG
Cron — ticket SLA breach	A	scheduler + delayed jobs	repeatable	schedule only	Ticket rows	next load / SSE	all data in PG
Permissions	B	perms:{tenant}:{user}	300s	cached perm list	UserAppRole / RolePermission /extras	sidebar/nav	re-query PG on miss
Profile (me)	B	employee-me:{tenant}:{user}	300s	cached profile	Employee	top bar	re-query PG on miss
Unread count	B	notif-unread:{tenant}:{user}	15s	cached count	Notification	bell badge	re-count PG on miss
Session re-validate	B	session-valid:{authUserId}	60s	auth verdict	central auth (HTTP)	n/a	re-checks central on miss (fail-open)
Central membership	B	central-member:{tenant}:{authUserId}	120s	membership verdict	central member API (HTTP)	n/a	re-checks central on miss (fail-open)
Active leave policy	B	leave-policy-active:{tenant}	300s	cached policy rules	LeavePolicy	leave apply screen	re-query PG on miss
Org-chart snapshot	B	org-chart:{tenant}	7 days	cached snapshot	Employee graph	org-chart page	recompute from PG on miss
Compensation analytics	B	analytics:compensation:{tenant}	36h	cached snapshot	EmployeeSalary SQL	analytics page	recompute from PG on miss
Birthdays / anniversaries / celebrations	B	birthdays: / anniversaries: / celebrations:{tenant}	300s	cached lists	Employee dates	home widgets	re-query PG on next load
Real-time updates (SSE)	C	channel realtime:{tenant}	none (signal)	nothing stored	the event's data is in PG	push → React Query refetch	browser re-fetches PG
Cross-instance cache bust	C	channel quikit:cache-invalidate	none (signal)	nothing stored	n/a	n/a	peers drop local copy

(Note: the rate limiter is **in-process memory**, not Redis — included here only to pre-empt the question.)

4. The honest gaps (so nothing is oversold)

The traces below are truthful about where behaviour is imperfect. The ones worth knowing up front:

- Bulk bank import does not use Redis/BullMQ at all** — it runs synchronously inside the HTTP request (up to ~5000 Employee.update s). The bulk-bank-import queue is a registered no-op placeholder.
- Bulk-import screen uses its own 2-second polling loop**, not SSE — a full page reload loses the in-progress import id.
- Repeatable crons only register in the standalone worker (worker/index.ts)**, not in the embedded worker — on an embedded-only/Vercel deploy, auto-close/SLA/analytics aren't scheduled (the /api/cron/* HTTP routes are the fallback).
- Org-chart drag-reassign invalidates the wrong React Query key** — the chart isn't refetched by that action (and even if it were, it could read the 7-day stale snapshot).

- **employee-me is not busted on profile edit** — an edited name/photo can be stale in the top bar for up to ~5 min.
- **New notifications don't bust** `notif-unread` — the badge can lag up to 15s (the short TTL is the only refresh).
- **Read endpoints never write back to Redis** — only the worker populates org-chart/analytics snapshots; without the worker, those reads recompute the heavy SQL every time (correct, just slower).
- **Caches with TTL-only eviction** (celebrations/birthdays/anniversaries) are never explicitly invalidated — editing a DOB shows up after ≤5 min.

5. What's in the rest of this document

- **Flow A** — BullMQ trigger flows: bulk import, payroll compute, PDF (button → queue → worker → Postgres → screen).
- **Flow B** — Email (the flagship example), ticket crons, analytics refresh, org-chart rebuild.
- **Flow C** — Read-through caches: permissions, profile, unread count, session/membership verdicts, leave policy (Redis → on miss → Postgres loader).
- **Flow D** — SSE real-time pipeline + the celebrations/anniversary caching (the second example), plus the home dashboard batch.
- **Flow E** — Client-side storage (`localStorage` / `sessionStorage`) + session management (NextAuth JWT cookie, central SSO, the shared Redis session id). Answers "do we store in localStorage or in memory?" and "do we manage sessions?".

Each feature is traced with: **Trigger** → **numbered code path (with file:line)** → **Redis-vs-Postgres split** → **TTL/expiry behaviour** → **how the screen updates** → **degraded mode without Redis**.

Flow A — BullMQ Background-Job Triggers (real code trace)

Traced from actual source on branch `gourav-uat-hrms`. Base dir: `apps/quikhrms`.

Shared infrastructure (read this first)

- **Enqueue helper:** `src/lib/queue/queues.ts`
 - `enqueue(queueName, jobName, data, opts)` → `getQueue(name).add(jobName, data, opts)`, returns `job.id`.
 - **Degraded mode:** if `queueConnection` is `null` (no `REDIS_URL`, or `REDIS_DISABLED=true`), `enqueue()` logs `[queue] skipped enqueue ...` and **returns ""** — **it no-ops, does NOT throw, does NOT run inline**. The triggering request still succeeds (e.g. the `DataImport` / `PayRun` row is created), but no worker ever picks it up, so status stays stuck at `ImportPending` / `queued` forever. There is **no inline fallback anywhere**.
 - `defaultJobOptions` (applied to every queue): `attempts: 3`, exponential backoff 5s, `removeOnComplete: { age: 3600, count: 100 }` (1h / last 100), `removeOnFail: { age: 172800 }` (2 days).
- **Redis connection:** `src/lib/queue/connection.ts` — BullMQ ioredis client; returns `null` when Redis is off (so importing the module never crashes a web route).
- **Queue names:** `src/lib/queue/types.ts` — `bulk-employee-import`, `bulk-bank-import`, `payroll-run`, `email`, `cron-tickets`, `pdf-generation`, `analytics`, `org-chart-rebuild`.
- **Worker registry:** `worker/processors/index.ts` — maps each queue → processor + concurrency.
- **Realtime / SSE:** publisher + subscriber in `src/lib/services/realtime.ts` (Redis Pub/Sub on channel `realtime:orgId`). SSE endpoint `src/app/api/v1/hrms/realtime/stream/route.ts` (GET, event: `<type>\ndata: <json>`). Client hook `src/lib/hooks/use-realtime.ts`, mounted app-wide via `RealtimeToastListener` + `ConnectionStatus` in `src/app/(dashboard)/hrms/layout.tsx:25-26`.
 - **Important:** the SSE pub/sub uses Redis **Pub/Sub**, which is separate from BullMQ. It is fire-and-forget (`.catch(() => {})`), carries no durable state, and has no TTL/fallback — if a client isn't connected at publish time, the event is lost. The durable status always lives in Postgres.

1. Bulk Employee Import — `bulk-employee-import` queue real BullMQ

Trigger

- Screen: `src/app/(dashboard)/hrms/employees/bulk-import/page.tsx`, "Import" / "Validate" button (`runImport()`), line ~854; button line ~1142).
- HTTP: `POST /api/v1/hrms/employees/bulk-import` with `{ fileName, rows, dryRun, markActive }`. Parsing (CSV/XLSX, column mapping, sub-sheets) happens **entirely client-side**; the server receives already-canonicalized `rows`.

Step-by-step flow

1. **Frontend** — `importMut.mutate({...})` posts the canonical rows (mutation at `page.tsx:367-369`).
2. **API handler** — `src/app/api/v1/hrms/employees/bulk-import/route.ts`
 - L12: Zod `bulkImportEmployeesSchema.safeParse`.
 - **L17-26: DB write at enqueue time** — `prisma.dataImport.create({ entityType:"employees", fileName, totalRows, status:"ImportPending", createdBy })`.
 - **L28-40: Redis write** — `enqueue("bulk-employee-import", "process-import", { orgId, triggeredByUserId, importId, fileName, rows, dryRun, markActive })`. The **entire row payload travels through Redis** (`BulkEmployeeImportJobData`, `types.ts:27-33`). Job inherits default `removeOnComplete 1h` / `removeOnFail 2d`.

- L42-46: returns `202 { importId, status:"queued", totalRows } .`
 - Rate-limited: 3 / 60s per tenant.
3. **Worker** — `worker/processors/bulk-employee-import.ts`
- L20-23: `dataImport.updateMany → status "ImportProcessing" .`
 - L25: re-validates rows with `bulkEmployeeRowSchema .`
 - L28: `processBulkEmployees(...)` (service `src/lib/services/gap-fill.ts`) does the actual employee inserts.
 - **L30-44: result persisted to Postgres** — `dataImport.update` with `processedRows/successRows/failedRows/errors` and final `status = ImportCompleted` (0 failed) / `ImportFailed` (0 success) / `ImportPartial` .
 - L47-51: `publishBulkImportProgress(...)` → SSE `bulk_import_progress` (fire-and-forget).
 - L53-63: audit log; if `!dryRun && success>0` → `scheduleOrgChartRebuild` (enqueues `org-chart-rebuild`).
 - L67-73: if `!dryRun` → `inviteImportedEmployees` (creates invitations + queues activation emails).

Redis vs Postgres

Redis (BullMQ job, ephemeral)	Postgres (durable)
Job payload incl. all rows , <code>dryRun</code> , <code>markActive</code>	<code>DataImport</code> row (status, totalRows, processedRows, successRows, failedRows, errors JSON, fileName, createdBy)
Retry/backoff state	<code>Employee</code> rows created by the service
Auto-removed: complete 1h / fail 2d	Invitations + queued emails (side effects)

How the screen updates

Client-side `setInterval` polling — NOT SSE, NOT React Query. `page.tsx:386-419`: after enqueue, `onSuccess` sets `pendingImportId`; a `useEffect` polls `GET /api/v1/hrms/employees/bulk-import/{importId}` (route `bulk-import/{importId}/route.ts`, reads `DataImport` from Postgres) **every 2000ms** until status is terminal (`ImportCompleted|ImportFailed|ImportPartial`), then renders the result card. A blue progress banner shows total/processed/ok/failed (`page.tsx:1160-1188`).

- The SSE `bulk_import_progress` event (worker L47) invalidates query key `["employees","bulk-import"]` (`use-realtime.ts:62-66`), but **this page does not use React Query for the import status**, so SSE has **no effect** here — the `setInterval` poll is the only live-update path.
- Banner text "Safe to leave page — refresh to resume polling" is accurate: polling only resumes if you re-open with the same `pendingImportId` in state (lost on full reload — see gap below).

When the Redis job is gone (after removeOnComplete/Fail)

No impact on the UI. All status is read from the `DataImport` table in Postgres via `GET /api/v1/hrms/employees/bulk-import/{importId}`. Redis only holds the in-flight job + retry state.

Degraded mode (no REDIS_URL)

`DataImport` row is created with `status:"ImportPending"`, `enqueue()` no-ops, **no worker runs**. The page polls forever and never reaches a terminal state — it stays on "Import queued...". No inline fallback.

2. Bulk Bank Import — `bulk-bank-import` queue ⚠️ NOT a background job (synchronous)

Surprising finding: despite the prompt's assumption, this feature does NOT use BullMQ at all. The `bulk-bank-import` queue name exists in `types.ts` and is registered in the worker as a **noop placeholder** (`worker/processors/index.ts:26`). **Nothing ever enqueues to it.** No Redis. No worker work. No `DataImport` row.

Trigger

- Screen: `src/app/(dashboard)/hrms/employees/bulk-bank-import/page.tsx`, "Import" button (`runImport()` L131).
- HTTP: `POST /api/v1/hrms/employees/bulk-bank-import` with `{ rows }` .

Step-by-step flow

1. **Frontend** — `importMut.mutate({ rows })` (`page.tsx:84-88`).
2. **API handler** — `src/app/api/v1/hrms/employees/bulk-bank-import/route.ts` — **does all work inline in the request:**
 - L24-26: Zod parse (`max 5000` rows).
 - L31-35: load matching `Employee` rows by `employeeCode` .
 - L40-65: **loop, one `prisma.employee.update` per row** setting `bankAccounts: [bankAccount]` (overwrites existing). Collects per-row errors.
 - L67-70: single audit log.
 - L72: returns `200 { total, updated, failed, errors }` synchronously.

Redis vs Postgres

Redis	Postgres
Nothing	<code>Employee.bankAccounts</code> updated in place; one <code>AuditLog</code> row

How the screen updates

Synchronous response → `onSuccess: setResult(res.data)` renders the result card immediately (`page.tsx:87`). No polling, no SSE, no refresh needed. Button shows "Importing..." while the single request is in flight.

When the Redis job is gone

N/A — nothing is ever written to Redis.

Degraded mode (no REDIS_URL)

Fully unaffected — it never touches Redis. Works regardless of Redis state. (This is the one feature that is robust to Redis being down, precisely because it's synchronous.)

Gap flagged: large bank imports (up to 5000 rows × 1 UPDATE each) run inline in a single HTTP request and can time out. The queue scaffolding exists but was never wired.

3. Payroll Run (compute) — `payroll-run` queue real BullMQ; job-STATE in Postgres

Trigger

- Screen: `src/app/(dashboard)/hrms/payroll/runs/[id]/page.tsx`, "Compute Payslips" / "Recompute" button (`computeMut.mutate()`), button line ~254).
- HTTP: `POST /api/v1/hrms/payroll/runs/{id}/compute` .

Step-by-step flow

- Frontend** — `computeMut` posts to the compute endpoint (`page.tsx:163-168`).
- API handler** — `src/app/api/v1/hrms/payroll/runs/[id]/compute/route.ts`
 - L14-18: load `PayRun` , must be `Draft` / `Processing` .
 - L20-23: **dedupe guard** — `getPayrollComputeState` ; if already `queued` / `running` → `409 conflict` .
 - L25-32: **DB write at enqueue time** — `setPayrollComputeState(orgId, id, { status:"queued" })` , which writes to the `PayRun.compute*` columns in **Postgres** (NOT Redis) — see `src/lib/services/payroll-compute-state.ts` (`computeStatus` , `computeStartedAt` , `computeFinishedAt` , `computeEmployeeCount` , `computeTotalGross/Net/Deductions` , `computeError`). State was folded out of a separate table into `PayRun` .
 - L34-38: **Redis write** — `enqueue("payroll-run", "compute", { orgId, triggeredByUserId, runId })` . **Payload is just `runId`** (`PayrollRunComputeJobData` , `types.ts:35-37`) — no payslip data in Redis.
 - L40: returns `202 { runId, status:"queued" }` .
- Worker** — `worker/processors/payroll-run.ts`
 - L19: rejects any job name ≠ `"compute"` .
 - L23-27: reload `PayRun` , re-check status.
 - L29-34: `setPayrollComputeState` → `"running"` (Postgres).
 - L37: `publishPayrollProgress(... status:"processing")` (SSE, fire-and-forget).
 - L40-41: `computePayslipsForRun` + `persistComputedPayslips` → **writes `Payslip` + `PayslipLine` rows to Postgres**.
 - L43-63: audit log + `emitPayrollEvent(RUN_PROCESSED)` .
 - L65-72: `setPayrollComputeState` → `"done"` with totals (Postgres).
 - L75-79: `publishPayrollProgress(... status:"done")` (SSE).
 - L86-98 (catch): `setPayrollComputeState` → `"failed"` with `error` (Postgres) + SSE `failed` ; rethrows so BullMQ marks the job failed/retries.

Redis vs Postgres

Redis	Postgres
BullMQ job (payload = <code>runId</code> only) + retry state	<code>PayRun.compute*</code> columns = the authoritative job state (queued/running/done/failed, started/finished, totals, error)
Pub/Sub <code>payroll_progress</code> events (ephemeral)	<code>Payslip</code> + <code>PayslipLine</code> rows (the actual computed result)
Auto-removed: complete 1h / fail 2d	<code>AuditLog</code> , <code>PayRun.status</code>

Confirmed as the prompt expected: **payroll job-state is in Postgres on `PayRun.compute*` , not Redis**. Redis only holds the BullMQ job (just the runId) and transient SSE pings.

How the screen updates

React Query polling + SSE assist (page.tsx:128-161):

- A useQuery(["payroll","runs",id,"compute-status"]) hits GET ../compute/status (route reads PayRun.compute* from Postgres). refetchInterval = 2000ms while status is queued|running , else false (stops polling at terminal state).
- When compute.status becomes done|failed , a useEffect invalidates ["payroll","runs",id] to refetch the run + pull the new payslips (L156-161).
- SSE adds a second nudge: payroll_progress events invalidate query key ["payroll"] (use-realtime.ts:56-60), so a connected client refreshes even between poll ticks. Either path works; the poll is the guaranteed one.
- UI shows "Queued.../Computing..." button states and a status banner with error/startedAt (L318-336).

When the Redis job is gone

No UI impact. Compute state is read from PayRun.compute* (Postgres) via GET ../compute/status ; payslips from the PayRun / Payslip tables. Redis losing the job only removes retry ability.

Degraded mode (no REDIS_URL)

setPayrollComputeState → "queued" is written to Postgres, enqueue() no-ops, no worker computes. The status endpoint returns queued forever, the page polls indefinitely, no payslips are produced. No inline fallback. (And because the 409 dedupe guard sees a stuck queued state, a retry is blocked until clearPayrollComputeState is called.)

4. PDF generation — pdf-generation queue real BullMQ (payslip-release only)

Finding: the pdf-generation queue is used ONLY for payslip release. Form 16 PDFs are generated synchronously in the request and never touch this queue (see note at bottom).

Trigger

- Screen: src/app/(dashboard)/hrms/payroll/runs/[id]/page.tsx , "Release Payslips" button (visible when run.status === "Approved" , line ~276-283) → handleRelease() → confirm modal → releaseMut.mutate() .
- HTTP: POST /api/v1/hrms/payroll/runs/{id}/release .
- Secondary trigger: per-payslip resend — POST ../runs/[id]/payslips/[payslipId]/resend-email also enqueues pdf-generation/payslip-release (resend-email/route.ts:35) .

Step-by-step flow

1. Frontend — releaseMut posts to the release endpoint (page.tsx:173-176).
2. API handler — src/app/api/v1/hrms/payroll/runs/[id]/release/route.ts
 - L17-20: assertTransition(status → "Paid") .
 - L24-78: \$transaction (all Postgres writes at request time) — process loan deductions (LoanRepayment create, EmployeeLoan update), payslip.updateMany → status:"Released", releasedAt , payRun.update → status:"Paid" , then collect released payslip ids.
 - L80-85: audit + emitPayrollEvent(RUN_RELEASED) .
 - L89-98: Redis writes — one job per payslip — enqueue("pdf-generation", "payslip-release", { orgId, triggeredByUserId, payslipId, payRunId }) (PayslipReleaseJobData , types.ts:57-60). PDF is NOT built here — only ids go to Redis.
 - L103: refreshLiabilityForPayRun (TDS, errors swallowed).
 - L107: returns 200 { id, status:"Paid", queuedPayslips } .
3. Worker — worker/processors/pdf-generation.ts (job-name dispatcher; payslip-release is the only handled case, L13; unknown names throw, L37).
 - Calls buildAndQueuePayslipEmail(...) → src/lib/services/payslip-release.ts :
 - loads payslip+lines, run, company, employee (Postgres);
 - builds the payslip PDF in the worker (buildPayslipPdf , L104);
 - queueEmail(...) → enqueues an email job with the PDF as a base64 attachment (L137-149) → emitted as PAYSZIP_EMAIL_SENT ;
 - returns { queued:false, reason } for skip cases (no email / not found) — logged, not a job error.

Redis vs Postgres

Redis	Postgres
pdf-generation job per payslip (payload = ids only)	Payslip.status="Released" , releasedAt ; PayRun.status="Paid"
email job carrying the generated PDF bytes (base64 attachment) — transient	LoanRepayment , EmployeeLoan updates; AuditLog ; TDS liability
Auto-removed: complete 1h / fail 2d	(the PDF itself is never persisted — it lives only as the email attachment)

How the screen updates

No live tracking of PDF/email jobs. On release success, releaseMut.onSuccess invalidates ["payroll","runs",id] once (page.tsx:175); the run flips to Paid and payslips show Released . The downstream PDF-build + email jobs run fully in the background with no UI feedback — the screen does not poll or show per-

payslip email status. `PAYSLIP_EMAIL_SENT/FAILED` are emitted as internal payroll events (not the SSE `payroll_progress` type), so the release screen does **not** surface them. **User must trust it / check email logs; flagged as a gap.**

When the Redis job is gone

The release outcome is durable in Postgres (`Payslip.status` , `releasedAt` , `PayRun.status="Paid"`). But the **PDF is only ever an email attachment — it's never stored**, so once the `pdf-generation` + `email` jobs are gone there is no PDF to re-read; re-sending requires the `resend-email` endpoint, which **rebuilds the PDF from scratch** off Postgres data.

Degraded mode (no REDIS_URL)

The release `$transaction` commits (run → Paid, payslips → Released) because it's inline, but `enqueue()` no-ops → **no PDFs built, no emails sent**. The UI still shows success. No inline fallback. Silent: employees simply never receive payslip emails.

Cross-feature summary

Feature	Real queue?	State of record	Live update mechanism	Redis-down behavior
Bulk employee import	✅ <code>bulk-employee-import</code>	<code>DataImport</code> (Postgres)	<code>setInterval</code> poll 2s (own fetch, not RQ/SSE)	enqueue no-op → stuck <code>ImportPending</code>
Bulk bank import	❌ none (noop placeholder)	<code>Employee.bankAccounts</code> (Postgres)	synchronous response	unaffected (never uses Redis)
Payroll compute	✅ <code>payroll-run</code>	<code>PayRun.compute*</code> cols (Postgres)	RQ <code>refetchInterval</code> 2s + SSE assist	enqueue no-op → stuck <code>queued</code> , dedupe blocks retry
Payslip release / PDF	✅ <code>pdf-generation</code> → <code>email</code>	<code>Payslip</code> / <code>PayRun</code> status (Postgres); PDF never persisted	none (one-shot invalidate; no job tracking)	enqueue no-op → no PDFs/emails, UI still "Paid"

Flagged gaps / surprises

- Bulk bank import is synchronous**, not a BullMQ job. The `bulk-bank-import` queue is a registered **no-op** with nothing enqueueing to it (`worker/processors/index.ts:26`).
- Form 16 PDFs are synchronous** (`form16/pdf/route.ts` , `form16/bulk-pdf/route.ts` call `buildPdf` inline and stream the bytes; never persisted, never queued). The `pdf-generation` queue handles **only** `payslip-release` .
- Bulk-import page uses raw `setInterval` polling**, not React Query and not the SSE stream — the `bulk_import_progress` SSE event it could consume has no effect on that page. Full page reload loses `pendingImportId` , so polling does not actually "resume" on refresh despite the banner copy.
- No inline fallback anywhere** when Redis is down — `enqueue()` silently returns `""` . Employee-import and payroll-compute both get permanently stuck in their pending/queued status; payslip release silently sends no emails.
- Payslip PDF is never stored** — it exists only as an email attachment inside a transient `email` job, so it vanishes with the job (1h after completion). Re-obtaining it requires `resend-email` , which regenerates from Postgres.
- Payroll dedupe trap**: a stuck `queued` state (Redis down) makes the compute endpoint return `409` , blocking retries until `clearPayrollComputeState` is run.

Flow B — Email, Cron, Analytics, Org-Chart (code + Redis end-to-end)

Traced from actual code under `apps/quikhrms` . All paths relative to that dir unless noted.

Shared infrastructure (read this first)

Two **separate** Redis clients exist — do not conflate them:

Client	File	Purpose	Config
Queue connection (BullMQ)	<code>src/lib/queue/connection.ts</code>	Job queues only	<code>maxRetriesPerRequest: null</code> , <code>enableReadyCheck: false</code> , <code>lazyConnect</code>
Cache client (<code>@quikit/redis</code>)	<code>packages/redis/index.ts</code>	<code>cacheGet/cacheSet/cacheDel</code> key-value cache (analytics + org-chart snapshots)	<code>maxRetriesPerRequest: 3</code> , <code>retry/backoff</code>

- Queue helper `enqueue()` (`src/lib/queue/queues.ts:37`): **no-ops** (returns `""`) when `queueConnection` is null — never crashes the request.
- Default job options (`queues.ts:5`): `attempts: 3` , exp backoff 5s, `removeOnComplete: { age: 3600, count: 100 }` (completed jobs drop from Redis after 1h / keep last 100), `removeOnFail: { age: 2d }` .
- Cache helpers (`packages/redis/index.ts:143-175`): all swallow errors and return null/no-op when `REDIS_URL` is unset → callers fall through to live compute.
- Worker is **embedded** in the Next process via `src/instrumentation.ts:13` → `src/lib/queue/embedded-worker.ts` (one worker set per process, guarded by a global flag). A standalone `worker/index.ts` also exists (Dockerfile entry).

⚠ Surprising gap #1 — schedulers only run in the standalone worker

`registerSchedulers()` (`worker/schedulers.ts`) is called **only** from `worker/index.ts:53` (the standalone/Docker worker). The **embedded worker** (`embedded-worker.ts`) starts the processors but **never calls** `registerSchedulers()` . So if the app runs only as the embedded Next process (e.g. Vercel with no separate worker container), the repeatable crons (`tickets-auto-close` , `tickets-sla-breach` , `compensation-all-tenants`) are **never registered**, and the nightly analytics/org-chart staleness window is covered only by manual triggers and the HTTP cron-fallback routes. Per-ticket delayed SLA jobs and on-demand enqueues still work (they don't need the scheduler).

FEATURE 1 — Email (flagship)

Is there a literal "email trigger button"?

No. There is no button in the HRMS UI whose sole purpose is "send email". Email is **always a side-effect** of a domain action. The closest things to an explicit button:

- **Invite / Resend invite** (`settings/users`) → `POST /api/v1/hrms/invitations` (and `/[id]/resend`) → invite email.
- **Approve / Reject** leave/WFH → decision email.
- **Recruitment "Send offer / Send interview / Send welcome"** routes (`/api/v1/hrms/mail/{offer,interview,welcome}`) — these are the most "button-like" email sends, surfaced from the recruit pipeline/interviews pages.

So tell the reviewer: the "email trigger button" is really the **Invite button** (and the recruit *Send* actions). The button saves a DB record, then enqueues an email job as a side-effect — there is no standalone "email" button.

Trigger

User clicks **Invite** in `settings/users` → `POST /api/v1/hrms/invitations` (`src/app/api/v1/hrms/invitations/route.ts:69`).

Step-by-step flow (invitation)

1. `POST /invitations` validates body, blocks duplicate employee/invite, validates roles (`route.ts:78-96`) — **Postgres reads**.
2. `provisionMemberRemote(...)` provisions the person centrally in QuikIT SSO (`route.ts:102`) — external HTTP, not Redis.
3. `prisma.invitation.create({...})` — **Postgres write**: the invitation RECORD (email, roleids, token hash, expiry, status Pending) is persisted (`route.ts:123`). Dual-write rollback via `deprovisionMemberRemote` if this fails (`route.ts:148`).
4. `dispatchInvitationEmail(...)` (`src/lib/services/invitation.ts:58`):
 - Builds subject+html from `buildInvitationEmail` (`invitation.ts:34`) — content is computed here, lives in the job payload.
 - `enqueue<EmailJobData>(QUEUE_NAMES.EMAIL, "invitation", {...})` (`invitation.ts:67`) → **Redis write**: `queue.add` pushes a job onto BullMQ list/stream for queue `email` . Payload = `EmailJobData` (`src/lib/queue/types.ts:45`): `{ orgId, to, subject, html, kind:"invitation" }` .
 - Returns `{ queued: true }` .
 - **Fallback** (`invitation.ts:75`): if `enqueue` throws (Redis down), it calls `sendMail()` **inline** (synchronous SMTP on the request thread) so a single invite is not lost. (Note: `enqueue` itself no-ops rather than throwing when `queueConnection` is null, so the inline fallback fires mainly on a real Redis *error*, not on a cleanly-unset `REDIS_URL` — see Degraded mode.)
5. `createAuditLog(...)` — **Postgres write** (audit row, records `emailQueued` / `emailSent`).
6. Response 201 `{ emailSent: mail.queued || mail.sent }` .
7. **Worker** (`worker/processors/email.ts:5`): `emailProcessor` reads `job.data` , decodes any base64 attachments, calls `sendMail()` (`src/lib/services/mailler.ts:86`) → **nodemailer SMTP send**. On success returns; the job is then dropped from Redis by `removeOnComplete` (1h). On failure it throws → BullMQ retries (3 attempts, exp backoff).

Other email triggers (all use the same `email` queue via `queueEmail`)

`queueEmail()` lives at `src/lib/services/mailler.ts:132` and enqueues an `EmailJobData` with a `kind` tag:

- **Ticket notifications** (`src/lib/services/ticket-notifications.ts:118` , `kind:"ticket.notify"`): created/assigned/status-change/comment/auto-closed/SLA-breach. Each first writes a `Notification` row + SSE publish (`notify()`) at `ticket-notifications.ts:74-121` , then queues email to recipients' `workEmail` .
- **Leave decision** (`src/app/api/v1/hrms/leaves/requests/[id]/approve/route.ts:148` , `kind:"leave.decision"`): after the approve/reject transaction commits, a fire-and-forget IIFE builds the decision email and `queueEmail(...)` .
- **WFH approve/reject** (`wfh/requests/[id]/approve|reject/route.ts`) — same pattern.
- **Payroll / payslip** (`src/lib/services/payroll-notifications.ts:68` , `kind:"payroll.notify"`): `RUN_RELEASED` etc. notify employees + admins; payslip PDF emails go through `src/lib/services/payslip-release.ts` .
- **Recruitment** mail routes (`/api/v1/hrms/mail/{offer,interview,welcome}`) and candidate-doc / interview-feedback crons (`/api/v1/hrms/cron/*`).
- **Bulk import auto-invite** (`src/lib/services/invitation.ts:113` `inviteImportedEmployees`): bulk variant `queueInvitationEmail` — **enqueue-only, no inline fallback** (so a 100-row import can never block on SMTP).

Redis vs Postgres split

Redis (transient)	Postgres (durable)
The email job on the <code>email</code> queue (subject+html+to in payload)	The Invitation / Notification / LeaveRequest / Payslip record
Retry state, removed after <code>removeOnComplete</code> (1h)	Audit log of <code>emailQueued</code> / <code>emailSent</code>
Nothing about the email persists after send	The domain record persists indefinitely (soft-delete)

The email **content is rendered into the job payload** (and the originating record lives in Postgres); the rendered HTML is **not** separately stored — once the worker sends via SMTP and the job completes, the payload is dropped from Redis.

TTL-expiry / removal behavior

- Completed job → removed after 1h (`removeOnComplete.age=3600`). Failed job → kept 2 days then removed.
- There is **no "resend on expiry"** of the job. If a job is lost (e.g. Redis flushed) before sending, the email is simply never sent — the Invitation row stays `Pending` ; recovery is the manual **Resend** button (`/api/v1/hrms/invitations/[id]/resend`). For ticket/leave/payroll emails there is no automatic re-send; the in-app `Notification` row (Postgres) is the durable record the user still sees.

How the screen updates

- Invite: React-Query mutation → toast "email sent" (driven by `emailSent` flag) → list refetch. No SSE for the email itself.
- Ticket/leave/payroll notifications: the **in-app** side is pushed via SSE (`publishNotification` / `publishTicketUpdate` in `realtime.ts`), independent of whether the email job has sent yet.

Degraded mode (no `REDIS_URL`)

- `queueConnection` is null → `enqueue` logs `[queue] skipped enqueue ...` and returns `""` (`queues.ts:44`). **No job, no inline send** for the `queueEmail` -based paths (ticket/leave/payroll/bulk-invite) → those emails are silently dropped (in-app Notification still written).
- The **single invite** path (`dispatchInvitationEmail`) is the exception: because `enqueue` no-ops instead of throwing, the `catch` inline-send fallback is actually only reached on a Redis *error*, not on cleanly-missing `REDIS_URL` . With `REDIS_URL` truly unset, the single invite would also be dropped unless SMTP-inline is reached — worth flagging as a subtle behavior.
- If SMTP itself is unconfigured, `sendMail` returns `{ sent:false }` (`mailer.ts:88`) and the worker job throws → retries then lands in the failed set.

FEATURE 2 — Cron: Tickets (auto-close + SLA breach)

Trigger

Two repeatable BullMQ schedulers on the `cron-tickets` queue (`worker/schedulers.ts:15`):

- `tickets-auto-close` — `0 3 * * *` (daily 03:00)
- `tickets-sla-breach` — `0 * * * *` (hourly)

Plus **per-ticket delayed jobs** scheduled at exact SLA due time, and **HTTP fallback** routes.

Step-by-step flow

Auto-close (`worker/processors/cron-tickets.ts:19` → `runAutoCloseSweep` in `src/lib/services/ticket-cron.ts:47`):

- Scheduler fires → BullMQ delivers job `tickets-auto-close` (Redis → worker).
- `prisma.ticketCategory.findMany({ autoCloseAfterDays > 0 })` — **Postgres read** (`ticket-cron.ts:48`).
- Per category, `prisma.ticket.findMany({ status:"Resolved", resolvedAt <= cutoff })` — **Postgres read** (`ticket-cron.ts:59`).
- `$transaction : ticket.updateMany → status "Closed" + ticketActivity.createMany ( AutoClosed )` — **Postgres writes** (`ticket-cron.ts:76`).
- `notifyTicketAutoClosed(...)` per ticket → in-app Notification (Postgres) + SSE + queued email (back onto the `email` queue).

SLA breach (`cron-tickets.ts:26` → `runSlaBreachSweep` , `ticket-cron.ts:118`):

- Scheduler/hourly job fires (Redis → worker).
- Three Postgres scans: response breaches (`slaResponseDueAt < now` , not yet breached), first resolve breaches (`slaResolveDueAt < now`), and ongoing escalations (`escalationLevel < 3`).
- Per breach: `$transaction` sets `responseBreachedAt` / `resolveBreachedAt` / `escalationLevel` , writes `ticketActivity` — **Postgres writes**.
- `getEscalationRecipients` (admins) → `notifyTicketSlaBreach` → Notification + SSE + queued email.

Per-ticket exact-time jobs (`src/lib/services/ticket-sla-scheduler.ts`):

- On ticket create (`tickets/route.ts:255`) and update (`tickets/[id]/route.ts:255`), `scheduleSlaCheck(orgId, ticketId, kind, dueAt)` enqueues a **delayed** job on `cron-tickets` with deterministic `jobId = sla:{kind}:{ticketId}` and `delay = dueAt - now + 30s` (`ticket-sla-scheduler.ts:56`). Re-scheduling replaces the prior delayed job (it `getJob` + `remove` first).
- Worker dispatches `SLA_JOB_NAMES.response|resolve` → `checkResponseSlaBreach` / `checkResolveSlaBreach` (`cron-tickets.ts:33-48`) — idempotent single-ticket version of the sweep, guarded by `responseBreachedAt` / `resolveBreachedAt` .
- `cancelSlaCheck` removes the delayed job when first response is recorded (`tickets/[id]/comments/route.ts:128`) or ticket closes.
- The **hourly sweep is the safety net** for any delayed job that was lost/missed.

HTTP fallback (`src/app/api/cron/tickets-auto-close/route.ts` , `.../tickets-sla-breach/route.ts`):

- `GET` guarded by `CRON_SECRET` (`x-cron-secret` header or Bearer), **fails CLOSED** if `CRON_SECRET` unset. Calls the same `runAutoCloseSweep` / `runSlaBreachSweep` directly — **no Redis needed**. Intended for Vercel Cron / manual re-run.

Redis vs Postgres split

Redis	Postgres
Repeatable schedule (cron pattern, next-run bookkeeping)	All ticket data, statuses, SLA timestamps, escalation level
Delayed per-ticket SLA jobs (<code>sla:{kind}:{ticketId}</code>)	<code>TicketActivity</code> audit rows, <code>Notification</code> rows
Nothing about the ticket lives in Redis	The breach/close decision is computed <i>from</i> Postgres and written to Postgres

TTL-expiry / removal behavior

- Repeatable jobs: `removeOnComplete:{count:100}` , `removeOnFail:{count:50}` (`schedulers.ts:69`). Delayed SLA jobs: `count:50` each.
- If a delayed SLA job is dropped, the **hourly sweep** still catches the breach (idempotent guards prevent double-processing). If the embedded worker never registered the cron (gap #1), the **HTTP fallback** route is the only recovery — Postgres is the source of truth either way.

How the screen updates

SSE only for the resulting in-app notification (`publishNotification` / `publishTicketUpdate`). The ticket list/detail page refetches via React-Query; there is no live push of the status change beyond the notification toast.

Degraded mode (no `REDIS_URL`)

- No queue → schedulers can't register and delayed jobs no-op. SLA/auto-close logic still runs **only** if the **HTTP cron routes** are hit by an external scheduler (they import the service directly and touch Postgres only).

FEATURE 3 — Analytics refresh (compensation)

Trigger

- Nightly scheduler `compensation-all-tenants` — `0 2 * * *` on the `analytics` queue (`worker/schedulers.ts:28`).
- Manual: `POST /api/v1/hrms/payroll/analytics/compensation/refresh` (enqueues `compensation-tenant`).

Step-by-step flow

1. Scheduler fires `compensation-all-tenants` (Redis → worker, `worker/processors/analytics.ts:18`).
2. **Master fan-out:** `prisma.companySettings.findMany({ select:{orgId} })` derives active tenants (no Tenant model) — **Postgres read** (`analytics.ts:20`). For each, `enqueue("compensation-tenant", {orgId})` — **Redis writes** (child jobs).
3. **Per-tenant child** (`analytics.ts:38`) → `refreshCompensationAnalytics(orgId)` (`src/lib/services/analytics-cache.ts:118`):
 - `computeCompensationAnalytics` runs 5 heavy `$queryRaw` aggregations over `EmployeeSalary` / `Employee` / `Department` / `OfficeLocation` — **Postgres reads** (`analytics-cache.ts:38-115`).
 - `cacheSet("analytics:compensation:{orgId}", JSON, 36*3600)` — **Redis write**, key `analytics:compensation:{orgId}` , **TTL 36h** (`analytics-cache.ts:122` , `TTL_SEC` at line 11).
4. **Read path** — `GET /api/v1/hrms/payroll/analytics/compensation` (`src/app/api/v1/hrms/payroll/analytics/compensation/route.ts:9`):
 - Unless `?fresh=1` , `readCompensationAnalytics(orgId)` → `cacheGet("analytics:compensation:{orgId}")` — **Redis read** (`route.ts:15`).
 - **On hit** → return cached JSON.
 - **On miss / null / expiry** → `computeCompensationAnalytics(orgId)` runs the live SQL and returns it (`route.ts:22`). **It does NOT write back to cache** — only the worker writes the cache. So a miss recomputes every request until the next nightly/manual refresh populates Redis.

Redis vs Postgres split

Redis	Postgres
<code>analytics:compensation:{orgId}</code> → JSON snapshot (<code>CompensationAnalytics</code>), TTL 36h	<code>EmployeeSalary</code> , <code>Employee</code> , <code>Department</code> , <code>OfficeLocation</code> (the raw data)
Pre-computed, best-effort	Source of truth; live SQL recompute on miss

TTL-expiry / removal behavior

- 36h TTL deliberately **survives one missed nightly run** (`analytics-cache.ts:11`). On expiry the endpoint **recomputes live** from Postgres each call (no write-back). `?fresh=1` forces a live recompute bypassing cache. There is no `cacheDel` for analytics — it just expires.

How the screen updates

Payroll page `Analytics` tab (`src/app/(dashboard)/hrms/payroll/page.tsx:98`) uses React-Query `compQ` against the compensation endpoint (enabled only when `tab === "Analytics"`). **Manual refetch / tab open** — no SSE. The screen shows whatever the endpoint returns (cache or live). There is **no UI button** wired to the `/refresh` POST endpoint (grep found no frontend caller) — refresh is API-only / nightly.

Degraded mode (no REDIS_URL)

cacheGet returns null → endpoint **always computes live** every request (correct but expensive). The nightly worker can't run anyway. Functionally correct, just no caching.

FEATURE 4 — Org-chart rebuild

Trigger

- scheduleOrgChartRebuild(orgId, reason, userId) (src/lib/queue/helpers.ts:10) enqueues org-chart-rebuild queue. Called from many write paths:
- Employee create / update / delete / restore: employees/route.ts:279 , employees/[id]/route.ts:221,275 , employees/[id]/restore/route.ts:16 .
 - Bulk delete / bulk-restore / bulk-hard-delete: employees/bulk-{delete,restore,hard-delete}/route.ts .
 - Bulk import (worker): worker/processors/bulk-employee-import.ts:62 (only when !dryRun && success>0).
 - Manual: POST /api/v1/hrms/org-chart/rebuild (org-chart/rebuild/route.ts:8 , returns 202 queued).
 - Drag-to-reassign in the org chart UI calls PATCH /employees/:id { reportingManagerId } (org-chart/page.tsx:216) which hits the employee-update route → which schedules a rebuild (reason:"employee.updated").

Step-by-step flow

- Write path calls scheduleOrgChartRebuild → enqueue<OrgChartRebuildJobData>(ORG_CHART_REBUILD, "rebuild", {orgId, reason}) — **Redis write** (helpers.ts:16). Fire-and-forget (void , errors logged).
- Worker** (worker/processors/org-chart-rebuild.ts:5) → refreshOrgChart(orgId) (src/lib/services/org-chart.ts:82):
 - computeOrgChart = prisma.employee.findMany({ where:{orgId, deletedAt:null}, select: ...appRoles, department, designation }) — **Postgres read** (org-chart.ts:37).
 - cacheSet("org-chart:{orgId}", JSON, 7*24*3600) — **Redis write**, key org-chart:{orgId} , **TTL 7 days** (org-chart.ts:84 , TTL_SEC line 12).
- Read path** — GET /api/v1/hrms/org-chart (src/app/api/v1/hrms/org-chart/route.ts:6):
 - Unless ?fresh=1 , readOrgChart → cacheGet("org-chart:{orgId}") — **Redis read**.
 - Hit** → return snapshot. **Miss / expiry** → computeOrgChart(orgId) live (route.ts:16). Again **no write-back** on the read path — only the worker (refreshOrgChart) or invalidateOrgChart touch the cache.
- invalidateOrgChart(orgId) = cacheDel("org-chart:{orgId}") exists (org-chart.ts:93) but grep found **no callers** — invalidation is done by re-enqueuing a rebuild, not by deleting the key.

Redis vs Postgres split

Redis	Postgres
org-chart:{orgId} → JSON OrgChartSnapshot , TTL 7d	Employee (+ appRoles , Department , Designation) — source of truth
Rebuild job on org-chart-rebuild queue	Reporting lines (reportingManagerId)

TTL-expiry / removal behavior

- 7-day TTL "covers stale window if a rebuild hook misses" (org-chart.ts:12). On expiry/miss, GET /org-chart recomputes live from Postgres (no write-back). So an employee change always reflects in the live recompute even if the rebuild job was lost; the cache just makes the common case fast.

How the screen updates

org-chart/page.tsx (useQuery(["org-chart-snapshot"], GET /org-chart) at line 290). After a drag-reassign, the mutation onSuccess does qc.invalidateQueries(["org-chart"]) (page.tsx:217) — **but the actual query key is ["org-chart-snapshot"]** , so the invalidation key mismatches the data query.

⚠ Surprising gap #2 — stale org chart after reassign

- The reassign mutation invalidates ["org-chart"] (page.tsx:217) but the org-chart data lives under ["org-chart-snapshot"] (page.tsx:291). The bulk-delete mutation *does* invalidate ["org-chart-snapshot"] correctly (page.tsx:1014,1046), but the drag-reassign path does not. Net effect: after a drag reassign, the PATCH succeeds and a rebuild is queued, but the on-screen chart is **not refetched** by that invalidation — it relies on the optimistic local state / a later refetch. Worth confirming against the rest of the file (lines >1199 not read), but the key mismatch is clear.
- Also note: the rebuild is async — even a correct refetch immediately after PATCH may read the **old cached snapshot** (7-day TTL) before the worker rebuilds it, unless the read used ?fresh=1 (the page does not). So the freshest data after a change comes from the next rebuild completing or a cache miss/expiry.

Degraded mode (no REDIS_URL)

scheduleOrgChartRebuild no-ops; cacheGet returns null → GET /org-chart **always computes live** from Postgres. Correct, just uncached (heavier per request).

Cross-cutting summary

- **Redis is never the source of truth** for any of these features. It holds: (a) transient job payloads (email, cron, rebuild, analytics fan-out), and (b) two best-effort cache snapshots (`analytics:compensation:{orgId}` 36h, `org-chart:{orgId}` 7d). Postgres holds every durable record.
- **Read endpoints always fall back to live Postgres compute on cache miss/expiry** — but **do not write back** (only the worker populates the cache). So without the worker running, reads are correct but uncached.
- **No literal "email button"** — email is a side-effect of Invite / Approve / recruit *Send* / cron actions. The reviewer's "email trigger button" maps to the **Invite** button (`POST /invitations`).
- **Two gaps worth raising**: (1) embedded worker doesn't `registerSchedulers()` → crons rely on the standalone worker or the `CRON_SECRET`-guarded HTTP routes; (2) org-chart drag-reassign invalidates the wrong React-Query key (`["org-chart"]` vs `["org-chart-snapshot"]`).

Flow C — Read-Through Caches (Code → Redis → Postgres)

Traced from actual source on branch `gourav-uat-hrms`. This documents the six read-through caches that ride the HRMS hot path: where each is read, the loader that hits Postgres on a miss, the TTL, what busts it, and the screen behind it.

0. The cache mechanism (read this first)

Files

- `apps/quikhrms/src/lib/services/cache.ts` — thin facade. `getCached(key, ttlSec, loader) → getOrSet ; invalidateKeys(...keys) → invalidate` per key. Also exports `cacheKeys` builders.
- `packages/auth/cache.ts` — the real layered cache (`getOrSet` , `invalidate`).
- `packages/redis/index.ts` — `@quikit/redis`: `getRedis()` (null when `REDIS_URL` unset), `cacheGet/cacheSet/cacheDel` (best-effort, swallow errors).

Layering — `getOrSet(key, ttlSeconds, loader)` (`packages/auth/cache.ts:147-175`)

The order is **LRU → Redis → loader**:

1. **Layer 1 — in-memory LRU** (`localGet` , `cache.ts:89-100`). `Map` of `{value, expiresAt}`, max 1000 entries, oldest-evicted. Per-process, instant, always on. If present and not expired → return immediately.
2. **Layer 2 — shared Redis** (`redisGet` → `cacheGet` , `cache.ts:111-119`). JSON-parsed. On hit, **backfills the LRU** with the same TTL (`localSet` , line 165) and returns.
3. **MISS — loader runs** (`cache.ts:170-173`). The loader is the only thing that touches Postgres (or central HTTP). Its result is written to the **LRU synchronously** (`localSet`) and to **Redis fire-and-forget** (`void redisSet`). Next call is hot.

`getOrSet` **fails open**: a Redis error in `redisGet` is caught and treated as a miss (`cache.ts:116-118`), so a broken cache degrades to "always run the loader," never an auth/data failure.

What "expiry" means at each layer

- **LRU**: `expiresAt = now + ttl*1000`. `localGet` deletes and returns `undefined` once `expiresAt < Date.now()` (`cache.ts:92-95`) → falls through to Redis.
- **Redis**: written with `SET key val EX ttlSeconds` (`packages/redis/index.ts:161`). Redis itself drops the key at TTL; `cacheGet` then returns `null` → treated as a miss → loader re-runs.
- **In all cases, expiry/miss = the loader re-runs the Postgres query and re-populates both layers**. This is the DB fallback.

Invalidation — `invalidate(key)` (`cache.ts:180-184`)

1. `localDelete(key)` — drop this process's LRU copy.
2. `await redisDelete(key)` — `DEL` the Redis key.
3. `await publishInvalidation(key)` — `PUBLISH quikit:cache-invalidate <key>`.

Every process lazily subscribes to `quikit:cache-invalidate` on first cache touch (`ensureInvalidationSubscriber` , `cache.ts:39-65`) and on receipt does `localStore.delete(key)` (line 59). **This is the cross-instance bust** — without it, peer pods would serve a stale LRU copy until their own TTL elapsed.

There is NO wildcard delete. Invalidation is exact-key only — every cached read must have a derivable invalidation key, or it can only age out by TTL.

Degraded mode (no Redis, `REDIS_URL` unset)

`getRedis()` returns `null` (`packages/redis/index.ts:59-64`); all Redis get/set/del/publish become no-ops, and the subscriber never starts. The cache collapses to **per-process LRU only**. Correctness is identical (loader still runs on miss); the only losses are cross-instance sharing and cross-instance invalidation — a peer pod ages out within the entry's TTL instead of being told to evict.

1. Permissions — `perms:{orgId}:{userId}` , TTL 300s

- **What is cached**: a `CachedPerms` object `{ roleCode, permissions: string[], mustChangePassword, preBoarding }` — the user's effective RBAC permission set. Key `cacheKeys.permissions(orgId, userId) = perms:{orgId}:{employeeId}` (`cache.ts:35`). TTL `PERM_CACHE_TTL_SEC = 300` (`with-auth.ts:215`).

- **Read flow** (`resolvePermissions` , `with-auth.ts:247-340`):

1. Every authenticated request → `withAuth` → `resolvePermissions(orgId, userId)` → `getCache(perms:..., 300, loader)` .
2. (a) LRU hit → return the perm object.
3. (b) Redis hit → backfill LRU → return.
4. (c) MISS → loader runs Postgres queries:
 - `prisma.employee.findFirst` (table **Employee**) for `mustChangePassword` + `status` (`with-auth.ts:252-259`).
 - `prisma.userAppRole.findMany` (table **UserAppRole** → joined **AppRole** → **RolePermission**), filtered to non-expired roles (`with-auth.ts:266-281`).
 - `prisma.userPermissionExtra.findMany` (table **UserPermissionExtra**) for per-user grant/deny (`with-auth.ts:282-286`).
 - Fallback to the tenant default `AppRole` if the user has no role rows (`with-auth.ts:292-302`).
 - Unions role perms + extras, applies DENYS last; `admin` → `["*"]` .
5. Result stored back into Redis + LRU.

- **On expiry / miss**: the loader re-runs the `UserAppRole/RolePermission/UserPermissionExtra` queries and re-populates both layers.

- **Invalidation** — `invalidatePermissionCache(orgId, userId?)` (`with-auth.ts:364-383`):

- Per-user (`userId` given): busts the one `perms:` key.
- Tenant-wide (`userId` omitted): **no wildcard** — enumerates every `UserAppRole.userId` in the tenant and busts each `perms:` key, chunked 100 at a time (`with-auth.ts:373-382`).
- Callers (write paths that change RBAC):
 - `settings/roles/[id]/route.ts:78,117` (role edit/delete) — tenant-wide.
 - `settings/roles/[id]/permissions/route.ts:46` , `settings/roles/[id]/navigation/route.ts:69` — tenant-wide.
 - `employees/[id]/role/route.ts:71` , `employees/bulk-role/route.ts:65` , `employees/[id]/permissions/route.ts:111` , `employees/[id]/confirm/route.ts:86` , `employees/[id]/temp-password/route.ts:42` , `employees/[id]/route.ts:208` — per-user.
 - `cron/preboarding-auto-activate/route.ts:44` — per-user (PreBoarding → Active widens perms).
 - `central-sync.ts:133` (central role transition) and logout (see §4) — per-user.
- Cross-instance bust: `invalidate` publishes each key on `quikit:cache-invalidate` ; peer pods drop their LRU copy.

- **Screen impact**: the **sidebar navigation** + any permission-gated UI. The browser reads perms from `GET /api/v1/hrms/dashboard/config` → that route returns `authCtx.permissions` straight from `withAuth` (`dashboard/config/route.ts:16,64`), and `useDashboardConfig` (`src/lib/hooks/use-dashboard-config.ts`) feeds `sidebar.tsx:278` (`hasAnyPermission` , `permissions.includes("*")`). So the perm cache backs which nav items render. **Stale-for-up-to-300s is acceptable** by design (the cache header in `packages/auth/cache.ts:11-13` states the trade-off); admin RBAC writes bust it immediately, and the client `useQuery` also has a 30s `staleTime` .

2. employee-me — `employee-me:{orgId}:{userId}` , TTL 300s

- **What is cached**: the caller's own profile card object (id, employeeCode, name, jobTitle, profilePhoto, status, reportingManagerId, department, designation, primary role). Key `cacheKeys.employeeMe` = `employee-me:{orgId}:{employeeId}` (`cache.ts:34`). TTL `300` (hardcoded at the call site, `employees/me/route.ts:13`).

- **Read flow** (`GET /api/v1/hrms/employees/me` , `employees/me/route.ts:9-48`):

1. Request → `withAuth` → `getCache(employee-me:..., 300, loader)` .
2. (a) LRU hit → return. (b) Redis hit → backfill LRU → return.
3. (c) MISS → loader: `resolveEmployeeId(orgId, userId)` then `prisma.employee.findFirst` (table **Employee**, joined **Department**, **Designation**, **UserAppRole**→**AppRole**) (`employees/me/route.ts:17-30`); shapes `role.{code,name,priority}` .
4. Stored back into Redis + LRU.

- **On expiry / miss**: the loader re-runs the `employee.findFirst` Postgres query and re-populates.

- **Invalidation**:

- Logout — `invalidateUserAuthCaches` busts `employee-me:` (`with-auth.ts:413`).
- Central-sync suspend/reactivate — `applyCentralState` busts `employee-me:` (`central-sync.ts:104,118`).
- **GAP / stale-screen risk**: ordinary profile edits do **not** appear to bust this key. `employees/[id]/route.ts` (PUT) busts only `perms:` (line 208), not `employee-me:` . So a name / photo / job-title / department change is stale in the top-bar + user menu for up to **300s** (plus the client's 5-min `staleTime`).

- **Screen impact**: the **top bar** (`top-bar.tsx:36-41` , `TopBar` greeting + `UserMenu` avatar/name/title) and `NotificationBell` 's owner data. Reads via `useQuery(["me","topbar"])` against `/employees/me` with `staleTime: 5*60_000` . A 300s-stale profile is mostly cosmetic, but combined with the missing invalidation it means an admin-edited or self-edited profile can lag visibly until both the server TTL and client `staleTime` lapse.

3. notif-unread — `notif-unread:{orgId}:{userId}` , TTL 15s (NOT 300)

- **What is cached**: an integer — the count of unread Notification rows for the caller. Key `cacheKeys.notifUnread` = `notif-unread:{orgId}:{employeeId}` (`cache.ts:36`). **TTL is 15** at the call site (`notifications/unread-count/route.ts:11`) — the prompt's "TTL unspecified" resolves to 15s, deliberately short "since SSE invalidates on new notification."

- **Read flow** (`GET /api/v1/hrms/notifications/unread-count` , `unread-count/route.ts:7-19`):

1. Request → `withAuth` → `getCache(notif-unread:..., 15, loader)`.
 2. (a) LRU hit / (b) Redis hit (backfill LRU) → return `{count}`.
 3. (c) MISS → loader: `prisma.notification.count({ where: { orgId, employeeId: userId, isRead: false } })` (table **Notification**).
 4. Stored back into Redis + LRU.
- **On expiry / miss:** the loader re-runs the `notification.count` query and re-populates.
 - **Invalidation** — busts `notif-unread`: only on the caller's own read-state changes:
 - `notifications/[id]/read/route.ts:19` (mark one read).
 - `notifications/read-all/route.ts:13` and `notifications/route.ts:60` (PATCH mark-all).
 - Logout — `invalidateUserAuthCaches` (`with-auth.ts:414`).
 - **GAP / stale-screen risk:** notification **creation** does NOT bust the recipient's key. `POST /api/v1/hrms/notifications` (`notifications/route.ts:39-51`) and the service-layer `prisma.notification.createMany` (e.g. `src/lib/services/task-notifications.ts:34`, plus payroll/ticket/form12bb/content-moderation notifiers) create rows with no `invalidateKeys`. A new notification is therefore invisible in the badge for up to **15s** (TTL only). The route comment claims "SSE invalidates on new notification," but the invalidation is the short TTL, not an explicit bust on create — the 15s TTL is what bounds the staleness.
 - **Screen impact:** the red **unread badge** on the bell (`top-bar.tsx:105-110,203-207`). Reads via `useQuery(["notifications","unread-count"])` with `staleTime: 60_000`. Marking read is optimistic on the client and busts the server key on settle; new-notification arrival relies on the 15s server TTL + the client refetch. Stale-for-up-to-15s on a count badge is acceptable.

4. session-valid — `session-valid:{authUserId}`, TTL 60s

- **What is cached:** an **auth verdict**, not data — a `boolean` (`true` = session still valid centrally). Key built inline `session-valid:${authUserId}` (`with-auth.ts:131`); `authUserId` is the central SSO subject (`token.id`), tenant-agnostic. TTL `SESSION_REVALIDATE_TTL_SEC = 60` (`with-auth.ts:20`).
- **Read flow** (`resolveIdentity`, `with-auth.ts:128-139`, runs only when `AUTH_URL` + `INTERNAL_SECRET` are set):
 1. Every request with a central JWT → `getCache(session-valid:..., 60, loader)`.
 2. (a) LRU hit / (b) Redis hit → return the cached boolean.
 3. (c) MISS → loader = `verifyTokenRemote({ authUrl, internalSecret, cookie })` (`packages/auth/verify-token-remote.ts:23-54`) — an **HTTP GET to central** `/api/verify-token`, NOT a local Postgres query. Maps `r.error → true` (**fail-open**: central unreachable/misconfigured does not lock everyone out) and `r.valid → true/false` (`false` = revoked/expired centrally).
 4. Verdict stored back into Redis + LRU. If `!stillValid` → `resolveIdentity` returns `null` → `withAuth` 401 → client bounces to `/login`.
- **On expiry / miss:** the loader re-runs the central `verifyTokenRemote` HTTP call and re-populates. The "DB fallback" here is the **central auth service** (HRMS has a separate DB and cannot read the central session table directly), accessed over HTTP and failing open.
- **Invalidation:**
 - Logout — `invalidateUserAuthCaches(authUserId, ...)` busts `session-valid: first` (`with-auth.ts:402`), so the very next request re-checks central auth instead of trusting the 60s window. This is the security-relevant bust (file comment, `with-auth.ts:388-392`).
 - Otherwise it self-heals within 60s (a central revoke propagates to HRMS within ≤60s).
 - Cross-instance: published on `quikit:cache-invalidate`.
- **Screen impact:** indirect — no widget reads it. A revoked/expired central session keeps working in HRMS for up to 60s, then any request 401s and the client redirects to `/login`. Stale-for-up-to-60s on an auth verdict is the documented trade-off (fail-open beats locking everyone out on a transient central outage).
- **Degraded (no Redis):** per-process LRU only; each pod independently re-verifies against central every 60s. Correctness identical; a logout bust only reaches the local process (peers age out in ≤60s).

5. central-member — `central-member:{orgId}:{authUserId}`, TTL 120s

- **What is cached:** a `boolean` membership-sync verdict (`true` = user still has live central HRMS access). Key `centralSyncCacheKey(orgId, authUserId) = central-member:{orgId}:{authUserId}` (`central-sync.ts:43-44`). TTL `CENTRAL_SYNC_TTL_SEC = 120` (`central-sync.ts:38`).
- **Read flow** (`syncCentralMembership`, `central-sync.ts:57-72`, called from `resolveIdentity`, `with-auth.ts:156`):
 1. Every request (after identity resolves) → `getCache(central-member:..., 120, loader)` (no-ops to `true` if `QUIKIT_URL` / `INTERNAL_SECRET` unset).
 2. (a) LRU hit / (b) Redis hit → return the verdict.
 3. (c) MISS → loader = `memberLookupRemote({ orgId, appSlug:"quikhrms", userIds:[authUserId] })` (`packages/auth/member-lookup-remote.ts:47-85`) — **HTTP POST to central** `/api/internal/members/lookup` (reads the LIVE central DB, since the JWT is stale). On `!lookup.ok` → returns `true` (**fail-open**). Otherwise `applyCentralState` (`central-sync.ts:79-143`) reconciles the local **Employee** row:
 - No live access → set Employee `status:"Suspended"` + `centralDeactivatedAt`, busts `perms:` and `employee-me:` (`central-sync.ts:98-105`), returns `false`.
 - Restored after sync-suspension → set `Active`, bust `perms:` + `employee-me:` (`central-sync.ts:113-120`).
 - Central role transition → `remapHrmsRole` (UserAppRole upsert/delete) + bust `perms:` (`central-sync.ts:131-133`).
 4. Verdict stored back into Redis + LRU. `false` → `resolveIdentity` returns `null` → 401 → `/login`.
- **On expiry / miss:** the loader re-runs the central `memberLookupRemote` HTTP call (and any reconciling Employee writes) and re-populates. The "DB fallback" is the **central QuikIT DB** over HTTP, not the local HRMS DB.

- **Invalidation:**
 - Logout — `invalidateUserAuthCaches` busts `centralSyncCacheKey(orgId, authUserId)` (`with-auth.ts:405`).
 - Otherwise self-heals within 120s (central removal/role change propagates within ≤120s).
 - Note: the verdict key itself isn't busted by the Employee writes inside `applyCentralState`; those write paths bust the *downstream* `perms: / employee-me:` keys so the new role/status is seen, while the sync verdict legitimately stays cached until its 120s TTL.
- **Screen impact:** indirect — a centrally-removed or app-access-revoked user keeps HRMS access for up to 120s, then 401s to `/login`. A central role change (member→org_admin) re-maps the HRMS role and busts `perms:`, so the sidebar reflects it on the next perms-cache miss. Stale-for-up-to-120s acceptable (same fail-open rationale as \$4).

6. Active leave policy — `leave-policy-active:{orgId}`, TTL 300s

Naming correction: the prompt calls this `active-policy:{orgId}`. The actual key in code is `leave-policy-active:${orgId}` (`leave-policy-engine.ts:25-27`).

- **What is cached:** an array of `CachedPolicy` candidates (approvedRules JSON + appliesTo filters + effective dates) for the tenant's Active leave policies. TTL `CACHE_TTL_SEC = 300` (`leave-policy-engine.ts:14`).
- **Read flow** (`resolveActivePolicyRules`, `leave-policy-engine.ts:432-459`, called from the leave-request create path before evaluating rules):
 1. Apply-leave submission → `getCached(leave-policy-active:..., 300, loader)`.
 2. (a) LRU hit / (b) Redis hit (backfill LRU) → return the candidate array.
 3. (c) MISS → loader = `loadCandidatePoliciesFromDb(orgId)` (`leave-policy-engine.ts:404-430`): `prisma.leavePolicy.findMany({ where: { orgId, status:"Active", deletedAt:null }, orderBy:[effectiveFrom desc, updatedAt desc] })` (table `LeavePolicy`).
 4. Stored back into Redis + LRU. Caller then filters by effective-date + dept/role/employment-type and `parseRules` the first match (`leave-policy-engine.ts:442-457`).
- **On expiry / miss:** the loader re-runs the `leavePolicy.findMany` Postgres query and re-populates. (Note: only the *policy candidates* are cached; the per-request balance/overlap/cap checks in `evaluateLeavePolicy` always hit Postgres fresh — `checkOverlap`, `checkBalance`, `checkMonthlyYearlyCaps`, `checkClubbing`, lines 221-390.)
- **Invalidation** — `invalidateLeavePolicyCache(orgId)` busts `leave-policy-active:{orgId}` (`leave-policy-engine.ts:29-31`). Callers (every policy write):
 - `leaves/policies/route.ts:74` (create).
 - `leaves/policies/[id]/route.ts:64,86` (update/delete).
 - `leaves/policies/[id]/approve/route.ts:41` (approve — makes it Active).
 - `leaves/policies/[id]/extract/route.ts:115` (AI rule extraction).
 - Cross-instance: published on `quikit:cache-invalidate`.
- **Screen impact:** no direct widget — it backs the **server-side leave-policy validation** on the Apply-Leave submit path (blocks/warns shown after submit). An admin editing a policy busts the cache immediately, so a new rule takes effect on the next submission; absent any write, a policy change made directly in the DB is stale for up to 300s. Acceptable — policy edits all go through the busting endpoints.

Summary table

#	Key	TTL	Loader (source)	Busted by
1	<code>perms:{tenant}:{emp}</code>	300s	Postgres: Employee + UserAppRole→AppRole→RolePermission + UserPermissionExtra	role/perm/nav edits, role assign, confirm, temp-pw, preboarding cron, central role change, logout
2	<code>employee-me:{tenant}:{emp}</code>	300s	Postgres: Employee (+Dept/Designation/Role)	logout, central suspend/reactivate — NOT profile edits
3	<code>notif-unread:{tenant}:{emp}</code>	15s	Postgres: <code>notification.count(isRead:false)</code>	mark-read, mark-all, logout — NOT notification create
4	<code>session-valid:{authUserId}</code>	60s	HTTP: central <code>/api/verify-token</code> (fail-open)	logout
5	<code>central-member:{tenant}:{authUserId}</code>	120s	HTTP: central <code>/api/internal/members/lookup</code> (fail-open)	logout
6	<code>leave-policy-active:{tenant}</code>	300s	Postgres: <code>LeavePolicy.findMany(status:Active)</code>	policy create/update/delete/approve/extract

All six share the same LRU→Redis→loader read path, the same fail-open behavior, and the same exact-key (no wildcard) pub/sub invalidation. Without Redis they all degrade to per-process LRU with identical correctness.

Flow D — SSE Realtime Pipeline + Celebrations / Dashboard Caching

Traced from actual code in `apps/quickhrms`. Two distinct Redis usages live here and the reviewer's "anniversary disappears after a TTL" observation maps to the **second** one, not the SSE bus.

Mechanism	Redis role	What's stored	TTL	Source of truth
SSE realtime (<code>realtime:{orgId}</code>)	Message bus (PUBLISH/SUBSCRIBE)	Nothing — fire-and-forget	n/a	Postgres (re-fetched after event)
Celebrations / birthdays / anniversaries / dashboard batch	Cache (SET with TTL)	Computed JSON list	300s (15s for notif count)	Postgres (loader on miss)

PART 1 — SSE Realtime Pipeline (Redis = transient message bus, nothing stored)

Trigger / data source

A backend write (asset change, leave approval, ticket update, payroll/bulk-import job progress, any notification) calls a `publish*` helper. Redis is used purely as a fan-out PUBLISH bus — **no key is written, no TTL, nothing persists**. The data the screen ultimately renders is re-fetched from Postgres via React Query invalidation.

Step-by-step flow

1. **Publish.** A route/service calls e.g. `publishNotification(...)` → `publishEvent(...)` → `pub.publish("realtime:{orgId}", JSON.stringify(event))`. `src/lib/services/realtime.ts:39-43` (publisher), helpers `:47-111`.

- Publisher is a lazily-created singleton `ioredis` client; in non-prod it's stashed on `globalThis.realtimePub` (`:21-36`). Throws if `REDIS_URL` unset (`:27`).
- This is a Redis PUBLISH only. Nothing is stored. No TTL.** (`:42`)

1. **SSE route subscribes.** `GET /api/v1/hrms/realtime/stream` (`src/app/api/v1/hrms/realtime/stream/route.ts`). `runtime = "nodejs"`, `dynamic = "force-dynamic"` (`:14-15`).

- Resolves identity from the session cookie via `resolveIdentity(req)` (`:19-30`);

dev no-login flow falls back to `x-tenant-id / orgId` query param (`:24-28`). 401 if neither (`:33-39`).

- Opens a `ReadableStream`; on `start` it emits `event: connected` (`:48`), then `createSubscriber(orgId, onMessage)` opens a **dedicated** `ioredis` connection and `SUBSCRIBE s realtime:{orgId}` (`realtime.ts:115-139`, called at `route.ts:60`).

1. **Per-employee filtering.** On each message, if `event.targetEmployeeIds` is non-empty and the connected `employeeId` isn't in it, the event is dropped (`route.ts:62-64`). Empty list = broadcast to whole tenant.

1. **Stream to browser.** Matching events are written as `event: {type}\ndata: {JSON payload}\n\n` (`route.ts:67-69`).

1. **30s heartbeat.** `setInterval(..., 30_000)` emits `event: heartbeat` to keep the connection alive (`route.ts:51-57`). Cleared on client disconnect / abort (`:54-55`, `:72-73`, `:78-82`).

1. **Cleanup.** `req.signal abort` and stream `cancel()` both call `subscriberRef.unsubscribe()` → `sub.unsubscribe()` + `sub.disconnect()` (`route.ts:78-86`, `realtime.ts:133-138`).

1. **Client hook.** `useRealtime` (`src/lib/hooks/use-realtime.ts`) opens an `EventSource` to the stream (`:41`), and on each typed event **invalidates a React Query key** then dispatches to registered handlers:

- `notification` → invalidate `["notifications"]` (`:50-54`)
- `payroll_progress` → invalidate `["payroll"]` (`:56-60`)
- `bulk_import_progress` → invalidate `["employees", "bulk-import"]` (`:62-66`)
- `ticket_update` → invalidate `["tickets"]` (`:68-72`)
- `asset_update` → invalidate `["assets"]` + `["asset", id]` (`:74-79`)
- `onerror` → close + exponential-backoff reconnect 1s→2s→...→max 30s (`:81-89`)

1. **Provider wiring.** `RealtimeProvider` (`src/components/hrms/realtime-provider.tsx`) wraps `useRealtime` in context; `enabled` only when session is `authenticated` (or dev ids present) (`:41-44`). Mounted app-wide in `src/components/providers.tsx:29`.

Which screens mount it / what they do on an event

- `RealtimeToastListener` (`src/components/hrms/realtime-toast.tsx`, mounted in `src/app/(dashboard)/hrms/layout.tsx:25`) — subscribes to `notification` and pops a toast (`success / warning / error / info`) (`:16-34`). No DB call itself.
- `Sidebar` (`src/components/hrms/layout/sidebar.tsx:305-320`) — on `notification`, invalidates `["notifications", "unread-count"]`, which refetches `/api/v1/hrms/notifications/unread-count` from Postgres. The badge is otherwise `staleTime: 60_000` with **no polling** (`:308-313`).

- `ConnectionStatus` (`src/components/hrms/connection-status.tsx:12`) — only reads

`status` , no data.

What the screen shows + how it updates

SSE is a **push-to-invalidate** model. The event payload itself is *not* rendered as data (except the toast text); it just tells React Query "this cache is stale," which triggers a **refetch from Postgres**. So: SSE push → `invalidateQueries` → component refetches the real list from the DB.

On event

The hook re-queries Postgres (via the invalidated query's `queryFn`). Redis stored nothing to expire — the event is a one-shot signal.

Degraded (no Redis)

- `getPublisher()` / `createSubscriber()` **throw** if `REDIS_URL` is unset (`realtime.ts:27,117`). Publishers in services are wrapped in `.catch(() => {})` (e.g. `payroll-notifications.ts:57`), so a write still succeeds; the realtime nudge is simply lost. The SSE route would 500 on connect; the client `onerror` keeps retrying.
- **Screens still work without SSE**: every consumer fetches via React Query with a `staleTime` , so data loads on mount/navigation/window-focus regardless. Only the *live push* (instant toast, instant badge bump) is lost — refresh/navigation recovers it.

PART 2 — Publishers (every call site)

Helper	Call site	Trigger
<code>publishAssetUpdate</code>	<code>assets/route.ts:144,159</code>	Asset created (bulk + single)
<code>publishAssetUpdate</code>	<code>assets/[id]/route.ts:54,74</code>	Asset updated / deleted
<code>publishAssetUpdate</code>	<code>assets/[id]/scrap/route.ts:75</code>	Asset scrapped
<code>publishAssetUpdate</code>	<code>assets/[id]/return/route.ts:99</code>	Asset returned
<code>publishAssetUpdate</code>	<code>assets/[id]/assign/route.ts:90</code>	Asset assigned
<code>publishNotification</code>	<code>delegations/route.ts:133</code>	Delegation created → notify recipients
<code>publishNotification</code>	<code>leaves/requests/[id]/approve/route.ts:100</code>	Leave approved → notify employee
<code>publishNotification</code>	<code>recruit/applications/[id]/route.ts:427</code>	Application stage change → notify
<code>publishNotification</code>	<code>lib/services/payroll-notifications.ts:52</code>	Payroll notification (e.g. payslip ready)
<code>publishNotification</code>	<code>lib/services/task-notifications.ts:48</code>	Task assignment / update
<code>publishNotification</code> + <code>publishTicketUpdate</code>	<code>lib/services/ticket-notifications.ts:93,99</code>	Helpdesk ticket status/assignment change
<code>publishPayrollProgress</code>	<code>worker/processors/payroll-run.ts:37,75,95</code>	BullMQ payroll worker: start / per-batch progress / failed
<code>publishBulkImportProgress</code>	<code>worker/processors/bulk-employee-import.ts:47</code>	BullMQ bulk-import worker: per-chunk progress

All publisher calls are fire-and-forget (`void` or `.catch(() => {})`).

PART 3 — Celebrations / Anniversaries / Birthdays — YES, Redis-cached with a TTL

This is what the reviewer saw. His assumption is **CORRECT** for this data.

Trigger / data source

The home widgets and the Celebrations calendar fetch employee birthday/joining dates. The API computes "upcoming" lists and **caches the result in Redis with a 300s (5-min) TTL** via the shared layered cache.

The cache primitive

`getCached(key, ttlSec, loader)` (`src/lib/services/cache.ts:12-14`) → `getOrSet` in `packages/auth/cache.ts:147-175` . Layered:

1. **In-memory LRU** (per-process Map, `cache.ts:80`) — first stop (`:158-159`).
2. **Shared Redis** (`@quikr/redis` , `cacheGet/cacheSet`) — second stop (`:162-167`),

backfills the local layer on hit.

1. **Loader** runs only on full miss; result written to *both* layers, Redis write is fire-and-forget (`:170-174`). `cacheSet` uses Redis `SET key value EX ttl` , so the key **physically expires and disappears from Redis after the TTL** — this is exactly the "sits with a TTL then is removed" behaviour seen in RedisInsight.

- **Fail-open:** any Redis error returns the loader's fresh value (`redisGet/redisSet` swallow errors, `:111-135`).

Cache keys + TTLs (all in HRMS)

Endpoint	Key	TTL	File:line
GET /employees/celebrations	celebrations:{orgId}	300s	employees/celebrations/route.ts:14
GET /employees/birthdays	birthdays:{orgId}	300s	employees/birthdays/route.ts:13
GET /employees/anniversaries	anniversaries:{orgId}	300s	employees/anniversaries/route.ts:14
GET /dashboard/batch (birthdays slice)	birthdays:{orgId}	300s	dashboard/batch/route.ts:40
GET /dashboard/batch (anniversaries slice)	anniversaries:{orgId}	300s	dashboard/batch/route.ts:57
GET /dashboard/batch (profile)	employee-me:{orgId}:{userId}	300s	dashboard/batch/route.ts:21
GET /dashboard/batch (notif count)	notif-unread:{orgId}:{userId}	15s	dashboard/batch/route.ts:35

The loader (e.g. `anniversaries/route.ts:16-42`) runs the Postgres `prisma.employee.findMany({ where: { orgId, deletedAt: null, status: "Active" }, ... })` , computes `daysUntil` / `years` , filters to the current month, sorts, slices.

What the screen shows + how it updates

- **Home page** (`src/app/(dashboard)/hrms/page.tsx`) mounts `BirthdaysWidget` / `AnniversariesWidget` (`dashboard-widgets.tsx:467-544`). Each is a React Query with `queryKey ["home", "birthdays"]` / `["home", "anniversaries"]` , `staleTime: 5*60_000` (`:469-473` , `:511-515`). **No SSE listener, no polling.**
- **Celebrations calendar** (`src/app/(dashboard)/hrms/celebrations/page.tsx:44-47`) —

React Query `["celebrations"]` , **no** `staleTime` (defaults to 0 → refetches on mount/focus), hits `/employees/celebrations` .

So the client has its own 5-min React-Query stale window *and* the server has a 5-min Redis TTL — two independent layers.

On TTL expiry — DOES the screen re-query the DB? Yes, on the next request.

The Redis key is removed at 300s, but **nothing pushes that to the screen**. The widget does not subscribe to SSE and does not poll. The chain is purely **pull**:

1. Redis key `anniversaries:{orgId}` expires/disappears at TTL → next API hit is a cache **miss**.

1. A miss only happens when the client actually re-requests — i.e. on ****page reload**, route re-navigation, window-refocus, or React-Query `staleTime` lapse** (5 min).

1. On that next request, `getOrSet` finds both layers empty → ****re-runs the Postgres loader****, recomputes the list, and re-populates Redis with a fresh 300s TTL.

Brutally honest correction to the reviewer's mental model: when the Redis key "disappears," the screen does **not** instantly react, re-call the DB, or blank out. The on-screen widget keeps showing its last fetched value until the *browser* decides to refetch (reload/focus/5-min stale). The DB re-query happens server-side on that next request, not at the moment the TTL fires. The TTL expiry he watched in RedisInsight and any on-screen change are **decoupled** — there is no expiry-driven push.

Also note: these three celebration caches are **never explicitly invalidated**. A grep for `invalidateKeys` shows busts only for `permissions` , `notif-unread` , `session-valid` , central-sync, and leave-policy keys — **not** `celebrations:` / `birthdays:` / `anniversaries:` . So editing an employee's DOB/joining date can stay stale for up to 5 minutes (TTL is the only eviction). That's an accepted staleness window, not a bug per the cache-strategy comment (`packages/auth/cache.ts:10-13`).

Degraded (no Redis)

- `getOrSet` still works: Layer 1 in-memory LRU serves within a process; on full miss

it runs the Postgres loader and returns fresh data. `isRedisCacheEnabled()` is just `Boolean(getRedis())` (`cache.ts:189`). So **celebrations/dashboard render fine with no Redis** — they degrade to per-process in-memory caching + direct Postgres reads.

- Cross-process invalidation (`quikit:cache-invalidate` channel) silently no-ops without Redis (`cache.ts:39-65`), but since these keys are never invalidated anyway, it's moot.

PART 4 — Home Dashboard Batch (`GET /api/v1/hrms/dashboard/batch`)

`src/app/api/v1/hrms/dashboard/batch/route.ts` — one request replacing 6+ calls. Runs 6 promises in parallel (`:19-108`):

1. **Profile** `employee-me:{orgId}:{userId}` — cached 300s (`:21-32`).
2. **Unread notif count** `notif-unread:{orgId}:{userId}` — cached **15s** (`:35-37`).
3. **Birthdays** `birthdays:{orgId}` — cached 300s (`:40-54`).
4. **Anniversaries** `anniversaries:{orgId}` — cached 300s (`:57-72`).

5. **Holidays** — **NOT cached**; direct `prisma.companyHoliday.findMany` each call

(:75-89).

1. **Attendance today** — **NOT cached** ("real-time", :91-107), direct Postgres read.

Client hook `useDashboardBatch` (`src/lib/hooks/use-dashboard-batch.ts:53-62`): `queryKey ["dashboard","batch"]`, `staleTime: 30_000`, `refetchOnWindowFocus: true`. Same pull model — refetches on focus / 30s stale; no SSE, no polling. On a Redis miss after TTL, the cached slices re-run their Postgres loaders.

Summary for the reviewer

- **SSE pipeline:** Redis is a pure **PUBLISH/SUBSCRIBE message bus** — nothing stored,

no TTL. Events trigger React-Query invalidation → screens **refetch from Postgres**. Heartbeat every 30s. Without Redis the live push is lost but screens still load via normal fetch.

- **Celebrations / birthdays / anniversaries / dashboard:** these ARE **Redis-cached

with a 5-min TTL (`getCached / getOrSet` → `SET ... EX 300`). The **"data appears, sits with a TTL, then is removed"** is real and is this cache. But **TTL expiry does not push anything to the screen and does not** by itself re-call the DB — the re-query happens on the *next* client request (reload/focus/5-min React-Query stale). The widgets have no SSE subscription and no polling. The Redis TTL and any visible refresh are decoupled. These keys are also never explicitly invalidated, so they're stale up to 5 min after an employee edit, by design.

Flow E — Client-Side Storage & Session Management (code-verified)

Added to answer three follow-up questions the Redis trace above does not cover: (1) what we use Redis for — answered throughout Flows A–D; (2) **do we store in localStorage or in memory?**; (3) **do we manage sessions?** This section traces the browser side (`LocalStorage` / `sessionStorage`) and the real session mechanism (NextAuth JWT cookie + central SSO + the shared Redis session id). Source-verified on branch `gourav-uat-hrms`, base dir `apps/quikhrms`.

Direct answers (one line each)

- **Where do we store data — localStorage or in memory?** Both, for *different, non-authoritative* things. The browser uses `localStorage` for device prefs + a small amount of dev/tenant scratch data, and `sessionStorage` for a *legacy* token pair. The server uses a per-process **in-memory LRU** as the first cache layer (see Flow C §0). **Neither browser storage nor the LRU is a source of truth — Postgres is.**
- **Do we manage sessions?** Yes — the session is a **NextAuth JWT held in an httpOnly cookie**, issued by **QuikIT's central SSO** (OAuth2/OIDC). It is *not* stored in `localStorage` / `sessionStorage`. A **shared Redis session id** lets a logout in HRMS (or any sibling app) soft-invalidate the session everywhere. The Redis `session-valid` / `central-member` verdicts in Flow C §4–5 sit on top of this.

PART 1 — Browser storage: `localStorage` and `sessionStorage`

`sessionStorage` — legacy per-tab token pair

`src/lib/auth/token-store.ts` keeps two values in `sessionStorage` (per-tab, not shared across tabs):

Key	What	Lifetime	File:line
<code>hrms_token</code>	short-lived access JWT (legacy Bearer header)	tab close	<code>token-store.ts:11,17,27</code>
<code>hrms_refresh</code>	long-lived refresh token (legacy <code>/auth/refresh</code>)	tab close	<code>token-store.ts:12</code>

Important: these are the **legacy** custom-auth tokens (`client-cleanup.ts:22` literally calls them "the legacy sessionStorage token pair"). The live auth path is the NextAuth cookie in PART 2 — **the real session is not in `sessionStorage`**. The token store is read/written via `getToken` / `setTokens` / `clearToken` and wiped on logout.

localStorage — device prefs + dev/tenant scratch

Key	What	Scope	Survives logout?	File:line
<code>hrms.theme</code>	dark/light dashboard theme	device pref	Yes (deliberate)	set <code>top-bar.tsx:307</code> ; read pre-hydration <code>app/layout.tsx:47</code>
<code>quikhrms-theme</code>	marketing-site theme	device pref	Yes	set <code>(marketing)/_components/nav.tsx:61</code> ; read <code>(marketing)/layout.tsx:39</code>
<code>hrms.sidebarCollapsed</code>	sidebar collapsed state	device pref	Yes (deliberate)	read <code>sidebar.tsx:285</code> ; set <code>sidebar.tsx:292</code>
<code>hrms.orgId</code>	dev no-login tenant id	dev identity	No — cleared	read <code>realtime-provider.tsx:25</code>
<code>hrms.userId</code>	dev no-login user id	dev identity	No — cleared	read <code>realtime-provider.tsx:26</code>
<code>hrms.roles</code>	dev role spoofing	dev identity	No — cleared	read <code>use-api.ts:16</code> , <code>payroll/runs/[id]/page.tsx:553</code>
<code>hrms.roles.impersonate</code>	dev role impersonation	dev identity	No — cleared	<code>client-cleanup.ts:11</code>
<code>hrms.asset.customCategories</code>	user-added asset categories (tenant data)	tenant scratch	No — cleared	read <code>assets/page.tsx:45</code> ; set <code>:225,238,267</code>

What is NOT in browser storage: no real session token, no permissions, no employee/profile data, no business records. The only tenant-scoped item is the asset-category scratch list, and it is explicitly cleared on logout so it can't leak to the next user on a shared machine.

Logout / expiry cleanup — `clearClientSessionState()`

`src/lib/auth/client-cleanup.ts:25-36` , called on logout and session expiry **before** navigating away:

- `abortAllInflight()` — cancels every in-flight GET so an old user's responses can't land.
- `clearToken()` — wipes the `sessionStorage` `hrms_token` / `hrms_refresh` pair.
- Removes the **session-scoped** `localStorage` keys (`SESSION_SCOPED_LOCAL_KEYS` , `client-cleanup.ts:9-15`): `hrms.roles` , `hrms.roles.impersonate` , `hrms.asset.customCategories` , `hrms.orgId` , `hrms.userId` .

Device prefs (`hrms.theme` , `hrms.sidebarCollapsed`) **deliberately persist** (`client-cleanup.ts:4-7`) — they carry no tenant data, and re-applying them on every login would be hostile UX.

PART 2 — Session management (NextAuth JWT cookie + central SSO)

Yes, HRMS manages sessions — but as an SSO client of QuikIT, not with its own login DB. Config: `src/lib/auth.ts` .

Where the session actually lives

- A NextAuth JWT in an httpOnly cookie**, signed with `NEXTAUTH_SECRET` . `session.strategy = "jwt"` , `maxAge` seven days (`auth.ts:92-93`). The cookie — not `localStorage` — is the session.
- Identity comes from **QuikIT's OAuth2/OIDC IdP** (`clientId` "quikhrms" , OIDC discovery via `wellKnown` , `id_token` verified) — `auth.ts:58-84` . HRMS has no local password/login table for SSO users.
- The JWT carries `id` (central user id), `email` , `orgId` , `membershipRole` , and a `sessionId` minted by the central IdP (`auth.ts:111-120`).

The `jwt` callback — soft-revocation every cycle (`auth.ts:95-122`)

- On initial sign-in, profile claims are copied onto the token.
- On later requests, throttled to `SESSION_CHECK_INTERVAL` (thirty seconds, `auth.ts:28,104`), it calls `isAuthSessionActive(token.sessionId)` against the **shared Redis session store** (`@quikit/auth/session-store`). If the session was revoked (central logout, admin force-logout, sibling-app sign-out) → returns `{}` , dropping all claims so middleware/ `withAuth` treat the request as unauthenticated.
- Fails open** when Redis is unavailable (a transient cache outage doesn't log everyone out).

The `signOut` event — one logout, everywhere (`auth.ts:135-160`)

- `revokeAuthSession(sessionId)` deletes the shared Redis session id → invalidates the session **centrally and across every sibling app** (without this, `/login` auto-SSO would silently sign the user straight back in).
- `invalidateUserAuthCaches(id, orgId)` busts this user's process-local auth caches (`session-valid` , `central-member` , `perms` , `employee-me`) so nothing lingers past the cookie.

Server-side enforcement (ties back to Flow C)

Every API request runs through `withAuth` → `resolveIdentity` (`src/lib/with-auth.ts`), which reads the JWT from the cookie via `getToken` , then applies the two Redis **verdict** caches already traced in Flow C:

- `session-valid:{authUserId}` (TTL sixty seconds, Flow C §4) — central session still valid? Re-checks central `/api/verify-token` on miss; fail-open.
- `central-member:{orgId}:{authUserId}` (TTL one-hundred-twenty seconds, Flow C §5) — central org membership / app access still live? Re-checks central member lookup on miss; fail-open; reconciles the local `Employee` row (suspend/reactivate/role remap).

So Redis only holds **boolean session verdicts**, never the session itself. The durable session is the cookie + the central SSO session record; HRMS is a stateless verifier.

Redis vs cookie vs browser-storage split

Layer	Holds	Source of truth?
httpOnly NextAuth cookie	the JWT session (id, orgId, sessionId, role)	yes (for the session)
Shared Redis session store	the sessionId liveness flag (cross-app revoke)	yes (for "is this session alive?")
Redis verdict caches	session-valid / central-member booleans, TTL-bounded	no (re-derivable from central)
sessionStorage	legacy hrms_token / hrms_refresh	no (legacy, not the live path)
localStorage	device prefs + dev identity + asset scratch	no

Degraded mode (no Redis)

The cookie session still works; isAuthSessionActive / session-valid / central-member all fail open, so users stay logged in and each request falls back to verifying against central over HTTP. The only loss is instant cross-app revocation — a central logout propagates within the verdict TTL (≤ one-hundred-twenty seconds) instead of immediately.

Summary for the reviewer

- **localStorage vs in-memory:** browser localStorage = device prefs (theme, sidebar) + small dev/tenant scratch; sessionStorage = a legacy token pair; server in-memory LRU = the hot first cache layer (Flow C §0). None is authoritative — Postgres is.
- **Sessions:** managed as a NextAuth JWT cookie via QuikIT central SSO, with a shared Redis session id for cross-app soft-revocation and two short-lived Redis verdict caches for server-side enforcement. The session is never kept in localStorage / sessionStorage .